Chương 6 – Quy hoạch động (Dynamic Programming): Mục 6.1–6.4

Oct 24, 2026 · @Hung Ta Quang

Quy hoạch động giải một bài toán lớn bằng cách chia nó thành các bài toán con lồng nhau, giải mỗi bài toán con **đúng một lần**, ghi kết quả vào bảng và dùng lại. Bốn mục 6.1–6.4 của giáo trình *Algorithm Design* (Kleinberg & Tardos) xây dựng kỹ thuật này theo từng nấc: 6.1 đi từ một hàm đệ quy, 6.2 rút ra nguyên lý chung, 6.3 mở rộng sang lựa chọn nhiều nhánh, 6.4 thêm một biến vào bài toán con.

Ghi chú nguồn: phiên làm việc này không có file sách đính kèm, nên nội dung được trình bày lại từ kiến thức về giáo trình. Ví dụ số và hình vẽ là minh hoạ tự dựng theo đúng tinh thần của sách; số thứ tự mệnh đề và hình nên đối chiếu với bản sách đang dùng.

Ý tưởng cốt lõi qua một hình ảnh

Hãy hình dung bạn leo một cầu thang n bậc, mỗi bước đi 1 hoặc 2 bậc, và muốn đếm số cách leo. Muốn đứng ở bậc n, bước cuối cùng phải xuất phát từ bậc n−1 hoặc bậc n−2. Vậy:

```
Cách(n) = Cách(n − 1) + Cách(n − 2)
```

Nếu mỗi lần cần "Cách(k)" ta lại tính từ đầu, cùng một bậc sẽ bị hỏi đi hỏi lại hàng trăm lần. Nếu **dán một mảnh giấy ghi kết quả lên từng bậc** ngay khi tính xong, ta chỉ cần đi một lượt từ dưới lên. Mảnh giấy đó chính là **bảng ghi nhớ** — linh hồn của quy hoạch động. Mục 6.1 sẽ gặp lại đúng hiện tượng này.

Vị trí của quy hoạch động so với hai kỹ thuật đã học

Kỹ thuật	Cách ra quyết định	Quan hệ giữa các bài toán con	Ví dụ trong sách
Tham lam (Chương 4)	Chọn ngay phương án trông tốt nhất, không quay lại	Chỉ còn một bài toán con sau mỗi lựa chọn	Lập lịch khoảng không trọng số, Dijkstra, MST
Chia để trị (Chương 5)	Chia đôi, giải độc lập, ghép kết quả	Các bài toán con **rời nhau**	Mergesort, đếm nghịch thế, cặp điểm gần nhất
Quy hoạch động (Chương 6)	Thử mọi khả năng cho phần tử cuối, nhưng nhớ kết quả	Các bài toán con **chồng lấn**, được dùng lại nhiều lần	Lập lịch có trọng số, bình phương tối thiểu từng đoạn, ba lô

Nói gọn: quy hoạch động là "vét cạn có trí nhớ". Nó xét mọi phương án như vét cạn, nhưng nhờ bảng ghi nhớ mà số phép tính chỉ còn đa thức.

6.1 Lập lịch khoảng có trọng số: một thủ tục đệ quy

Mục này cho thấy một bài toán mà tham lam thất bại vẫn giải được trong thời gian O(n) (sau khi sắp xếp), chỉ nhờ một công thức truy hồi hai nhánh và một mảng ghi nhớ.

Phát biểu bài toán

Có n yêu cầu sử dụng một tài nguyên duy nhất. Yêu cầu i chiếm khoảng thời gian $[s_i, f_i)$ và mang lại giá trị $v_i > 0$. Hai yêu cầu **tương thích** nếu khoảng của chúng không chồng lấn. Mục tiêu: chọn một tập con các yêu cầu đôi một tương thích sao cho **tổng giá trị lớn nhất**.

Hình ảnh đời thường: một hội trường duy nhất, nhiều đơn vị đăng ký các khung giờ khác nhau, mỗi đơn vị trả một mức phí khác nhau. Người quản lý muốn xếp lịch để doanh thu cao nhất. Trong lĩnh vực thủy lợi, có thể nghĩ đến một trạm bơm chỉ phục vụ được một khu tưới tại mỗi thời điểm, mỗi khu xin một khung giờ và mang lại một lợi ích khác nhau.

Vì sao tham lam không còn đúng

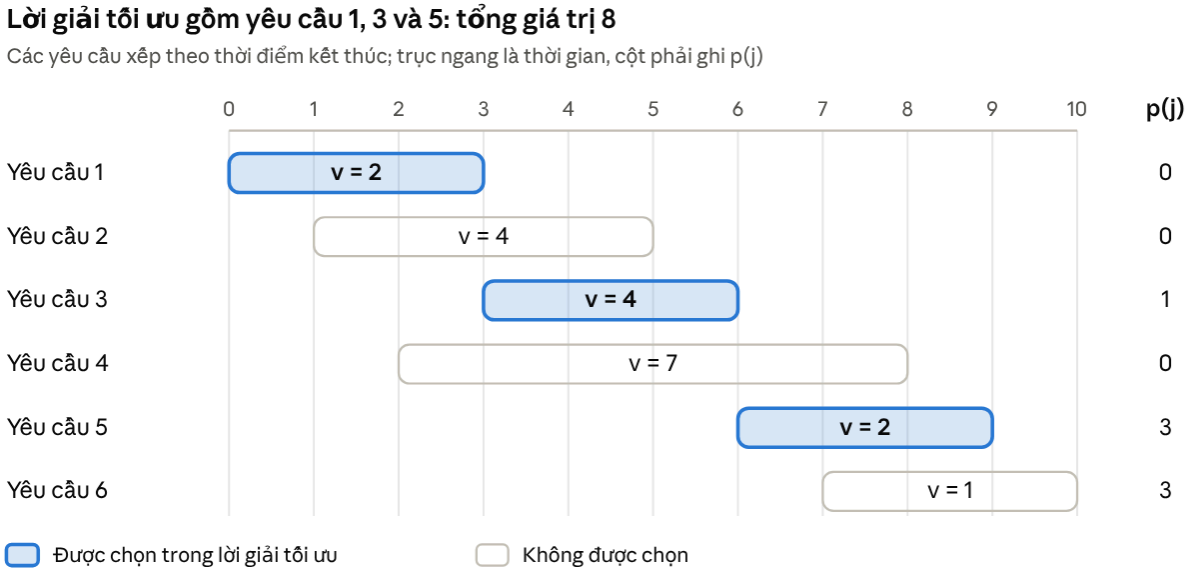
Ở Chương 4, khi mọi $v_i = 1$, quy tắc "chọn yêu cầu kết thúc sớm nhất" cho lời giải tối ưu. Khi có trọng số, quy tắc này sụp đổ ngay với hai yêu cầu: A = [0, 2) giá trị 1 và B = [1, 10) giá trị 100. Tham lam chọn A vì A kết thúc sớm, rồi không chọn được B nữa: thu 1 thay vì 100. Không có quy tắc tham lam tự nhiên nào đúng cho bài toán này, nên ta cần một ý tưởng khác.

Hai bước chuẩn bị

1. **Sắp xếp** các yêu cầu theo thời điểm kết thúc tăng dần: $f_1 \leqslant f_2 \leqslant \ldots \leqslant f_n$.
2. **Định nghĩa p(j)**: chỉ số i < j lớn nhất sao cho yêu cầu i tương thích với j (tức $f_i \leqslant s_j$). Nếu không có i nào như vậy thì p(j) = 0. Có thể hiểu p(j) là "yêu cầu gần nhất phía trước mà j còn chung sống được".

Ví dụ chạy xuyên suốt mục này gồm 6 yêu cầu:

j	Khoảng $[s_j, f_j)$	Giá trị v_j	p(j)
1	[0, 3)	2	0
2	[1, 5)	4	0
3	[3, 6)	4	1
4	[2, 8)	7	0
5	[6, 9)	2	3
6	[7, 10)	1	3



Hình 6.1a · sáu yêu cầu có trọng số và p(j)

Đọc hình: p(j) là yêu cầu có chỉ số lớn nhất kết thúc trước khi j bắt đầu. Chẳng hạn yêu cầu 5 bắt đầu lúc 6, đúng lúc yêu cầu 3 kết thúc, nên p(5) = 3.

Quan sát then chốt: yêu cầu cuối có được chọn hay không

Gọi O là một lời giải tối ưu và OPT(j) là giá trị tối ưu khi chỉ xét các yêu cầu 1, 2, …, j (quy ước OPT(0) = 0). Với yêu cầu cuối n, chỉ có hai khả năng:

- **$n \in O$.** Khi đó mọi yêu cầu p(n)+1, …, n−1 đều chồng lấn với n nên bị loại. Phần còn lại của O phải là lời giải tối ưu cho {1, …, p(n)}; nếu không, ta thay bằng lời giải tốt hơn và được O tốt hơn — mâu thuẫn. Giá trị: v_n + OPT(p(n)).
- **$n \notin O$.** Khi đó O chính là lời giải tối ưu cho {1, …, n−1}. Giá trị: OPT(n−1).

Vì không biết trước khả năng nào xảy ra, ta lấy cái tốt hơn. Áp dụng cho mọi j, ta có **công thức truy hồi trung tâm** của mục:

```
OPT(j) = max( v_j + OPT(p(j)) ,  OPT(j − 1) )        với OPT(0) = 0
```

Đi kèm là **quy tắc nhận diện**: yêu cầu j thuộc một lời giải tối ưu của {1, …, j} khi và chỉ khi v_j + OPT(p(j)) ⩾ OPT(j−1).

Hình ảnh: đứng trước yêu cầu j giống như đứng trước hai cánh cửa. Cửa "nhận" dẫn bạn nhảy lùi về phòng p(j) và cộng thêm v_j; cửa "bỏ" dẫn bạn lùi một bước về phòng j−1. Bạn mở cả hai cửa, xem phòng nào có giá trị lớn hơn rồi mới quyết định.

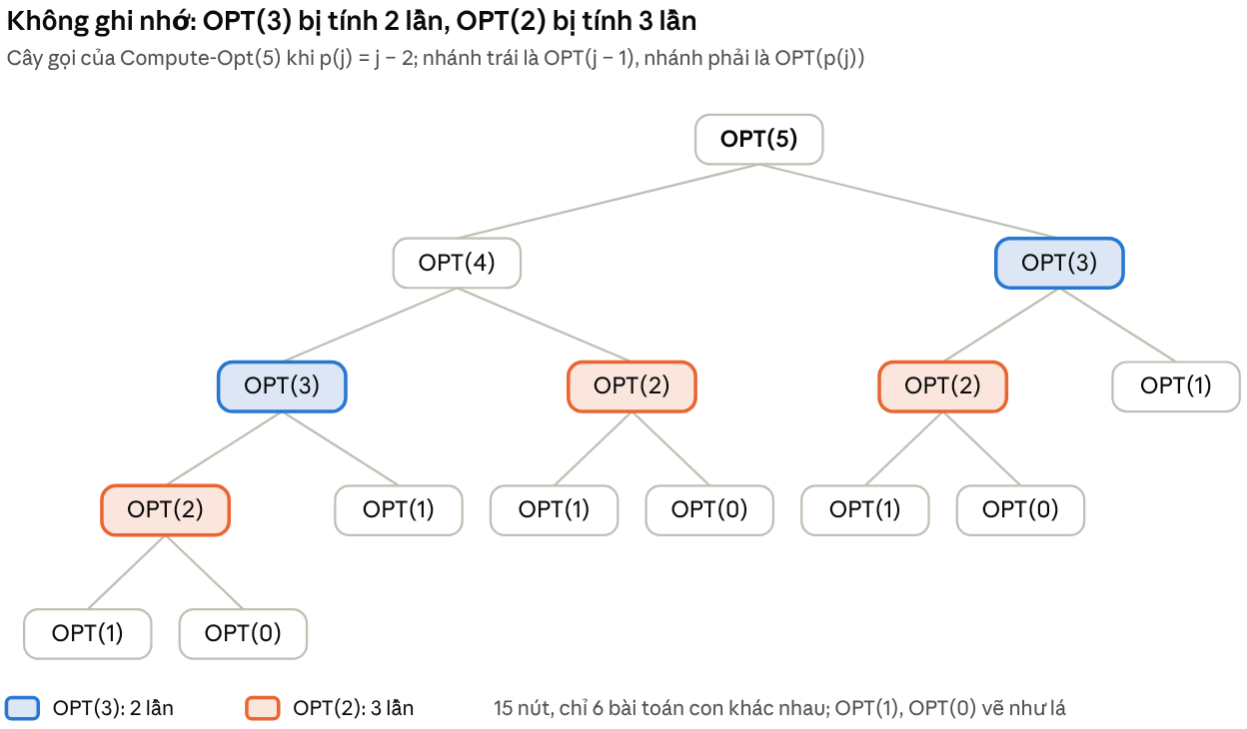
Thủ tục đệ quy Compute-Opt và sự bùng nổ

Công thức trên chuyển thẳng thành mã:

```
Compute-Opt(j)
  if j = 0 then return 0
  else return max( v[j] + Compute-Opt(p[j]),  Compute-Opt(j-1) )
```

Tính đúng được chứng minh bằng quy nạp theo j: với j = 0 hiển nhiên; giả sử Compute-Opt(i) = OPT(i) với mọi i < j, thì vì p(j) < j và j−1 < j, giá trị trả về đúng bằng vế phải của công thức truy hồi, tức OPT(j).

Vấn đề nằm ở **thời gian**. Mỗi lời gọi sinh ra hai lời gọi con. Xét trường hợp xấu p(j) = j−2: số lời gọi cho Compute-Opt(n) tăng như dãy Fibonacci, khoảng $1{,}618^n$ — hàm mũ. Hình dưới cho thấy nguyên nhân: cùng một bài toán con bị tính lại nhiều lần.



Hình 6.1b · cây gọi đệ quy của Compute-Opt(5)

Mỗi tầng sâu thêm, số nút gần như nhân đôi, trong khi số bài toán con khác nhau chỉ tăng thêm một. Đó là dấu hiệu nhận biết kinh điển của bài toán cần quy hoạch động: các bài toán con chồng lấn.

Điều đáng chú ý là cây gọi có hàm mũ nút, nhưng chỉ có **n+1 bài toán con khác nhau**: OPT(0), OPT(1), …, OPT(n). Toàn bộ sự lãng phí đến từ việc tính lại.

Ghi nhớ (memoization): M-Compute-Opt

Ta dùng một mảng M[0..n], ban đầu rỗng, M[0] = 0. Lần đầu tính OPT(j) thì ghi vào M[j]; những lần sau chỉ việc đọc ra.

```
M-Compute-Opt(j)
  if j = 0 then return 0
  else if M[j] đã có giá trị then return M[j]
  else
    M[j] ← max( v[j] + M-Compute-Opt(p[j]),  M-Compute-Opt(j-1) )
    return M[j]
```

Phân tích thời gian. Dùng "thước đo tiến độ" là số ô của M đã được điền. Mỗi lời gọi hoặc trả ngay một ô có sẵn trong O(1), hoặc điền thêm một ô mới và sinh tối đa hai lời gọi con. Vì M chỉ có n ô cần điền, số lần điền mới tối đa là n, kéo theo tổng số lời gọi tối đa khoảng 2n+1. Vậy M-Compute-Opt(n) chạy trong **O(n)**, với giả thiết đầu vào đã sắp xếp và đã có p(·). Bước sắp xếp và tính p(·) (bằng tìm kiếm nhị phân) tốn O(n log n).

Truy vết lời giải: Find-Solution

Mảng M chỉ cho **giá trị** tối ưu. Để biết **chọn những yêu cầu nào**, ta đi ngược từ M[n] và dùng quy tắc nhận diện ở trên:

```
Find-Solution(j)
  if j = 0 then dừng
  else if v[j] + M[p[j]] ≥ M[j-1] then
    xuất j;  Find-Solution(p[j])
  else
    Find-Solution(j-1)
```

Mỗi lời gọi giảm j ít nhất 1 nên Find-Solution chạy trong O(n). Không cần lưu thêm bảng phụ nào: bảng M đã chứa đủ thông tin để dựng lại lời giải.

Chạy tay trên ví dụ

j	v_j + M[p(j)]	M[j−1]	M[j]	Quyết định
0	—	—	0	cơ sở
1	2 + M[0] = 2	0	2	nhận
2	4 + M[0] = 4	2	4	nhận
3	4 + M[1] = 6	4	6	nhận
4	7 + M[0] = 7	6	7	nhận
5	2 + M[3] = 8	7	8	nhận
6	1 + M[3] = 7	8	8	bỏ

Truy vết từ j = 6: 7 < 8 nên bỏ 6, lùi về 5. Tại 5: 8 ≥ 7 nên **nhận 5**, nhảy về p(5) = 3. Tại 3: 6 ≥ 4 nên **nhận 3**, nhảy về p(3) = 1. Tại 1: 2 ≥ 0 nên **nhận 1**, nhảy về 0 và dừng. Lời giải tối ưu là {1, 3, 5} với tổng giá trị 2 + 4 + 2 = **8**. Lưu ý yêu cầu 4 có giá trị lớn nhất (7) nhưng không được chọn, vì nó chặn mất các tổ hợp tốt hơn — đúng là kiểu bẫy mà tham lam rơi vào.

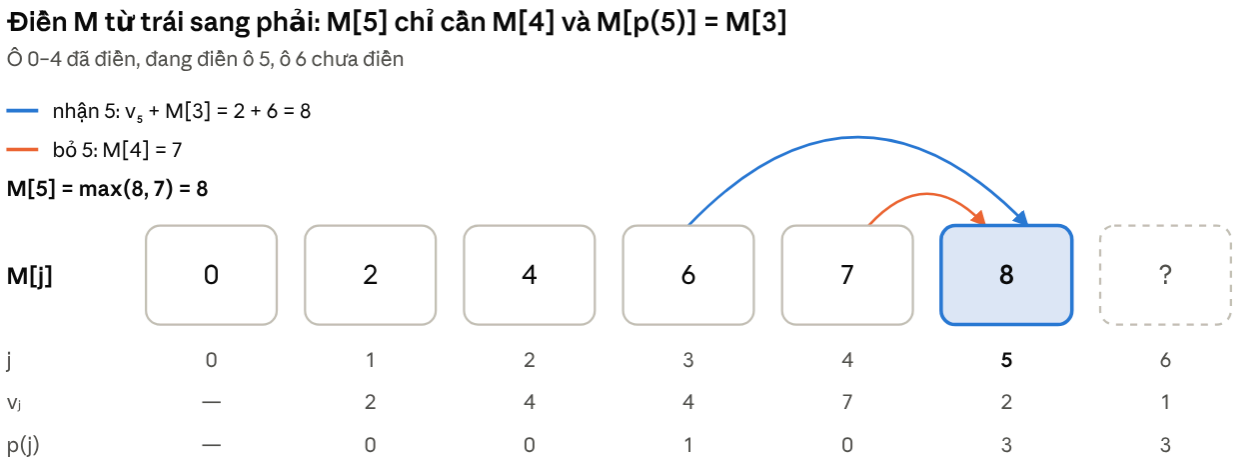
6.2 Nguyên lý quy hoạch động: ghi nhớ hay lặp trên các bài toán con

Điều cốt lõi của 6.1 không phải là phép đệ quy mà là **mảng M**: một khi biết M[j] chỉ phụ thuộc vào các ô có chỉ số nhỏ hơn, ta có thể bỏ đệ quy và điền M bằng một vòng lặp từ trái sang phải. Đây là cách sách khuyến nghị trình bày các thuật toán quy hoạch động ở phần còn lại của chương.

Từ đệ quy sang vòng lặp

```
Iterative-Compute-Opt
  M[0] ← 0
  for j = 1 to n
    M[j] ← max( v[j] + M[p[j]],  M[j-1] )
```

Vì p(j) < j và j−1 < j, khi tới ô j thì hai ô cần dùng chắc chắn đã được điền. Vòng lặp chạy n lần, mỗi lần O(1), nên tổng thời gian **O(n)** — bằng phiên bản ghi nhớ, nhưng không tốn ngăn xếp đệ quy. Tính đúng chứng minh bằng quy nạp theo j, giống hệt Compute-Opt.



Hình 6.2 · mảng M được điền dưới lên

Hình ảnh: mảng M như một dãy hộp được đổ đầy từ trái sang phải. Muốn đổ hộp j, ta chỉ nhìn vào hộp ngay trước (bỏ j) và hộp p(j) (nhận j) — cả hai đều đã đầy.

Cài đặt Python đầy đủ cho bài toán 6.1, gồm sắp xếp, tính p(j) bằng tìm kiếm nhị phân, điền bảng và truy vết:

```
from bisect import bisect_right

def weighted_interval_scheduling(jobs):
    """jobs: danh sách (s, f, v). Trả về (giá trị tối ưu, các yêu cầu được
chọn)."""
    jobs = sorted(jobs, key=lambda x: x[1])          # sắp theo thời điểm kết
thúc
    n = len(jobs)
    finish = [f for _, f, _ in jobs]

    # p[j] (đánh số từ 1): số yêu cầu có f <= s_j  -> chỉ số lớn nhất tương
thích
    p = [0] * (n + 1)
    for j in range(1, n + 1):
        s_j = jobs[j - 1][0]
```

```
        p[j] = bisect_right(finish, s_j, 0, j - 1)

    M = [0] * (n + 1)
    for j in range(1, n + 1):
        v_j = jobs[j - 1][2]
        M[j] = max(v_j + M[p[j]], M[j - 1])

    # truy vết
    chosen, j = [], n
    while j > 0:
        v_j = jobs[j - 1][2]
        if v_j + M[p[j]] >= M[j - 1]:
            chosen.append(jobs[j - 1]); j = p[j]
        else:
            j -= 1
    return M[n], chosen[::-1]

vi_du = [(0, 3, 2), (1, 5, 4), (3, 6, 4), (2, 8, 7), (6, 9, 2), (7, 10, 1)]
print(weighted_interval_scheduling(vi_du))
# (8, [(0, 3, 2), (3, 6, 4), (6, 9, 2)])
```

Hai phong cách: trên xuống và đưới lên

Tiêu chí	Ghi nhớ (trên xuống)	Lặp (dưới lên)
Xuất phát	Từ bài toán gốc OPT(n), đệ quy xuống	Từ bài toán nhỏ nhất OPT(0), đi lên
Thứ tự tính	Do lời gọi đệ quy tự quyết định	Người lập trình chỉ định rõ
Ưu điểm	Viết gần như nguyên văn công thức; chỉ tính các bài toán con thực sự cần	Không tốn ngăn xếp đệ quy; dễ phân tích thời gian; dễ tiết kiệm bộ nhớ
Nhược điểm	Có thể tràn ngăn xếp khi n lớn	Phải tìm ra thứ tự điền hợp lệ

Hai cách có cùng độ phức tạp và cho cùng bảng M. Sách dùng phong cách lặp cho các thuật toán sau, nhưng mọi thuật toán đều có thể viết lại theo kiểu ghi nhớ.

Ba điều kiện của một tập bài toán con tốt

Để phát triển một thuật toán quy hoạch động, ta cần tìm một tập bài toán con thoả mãn:

1. **Số lượng đa thức.** Chỉ có đa thức bài toán con khác nhau (ở 6.1 là n+1).
2. **Dễ suy ra lời giải gốc.** Lời giải bài toán gốc tính được dễ dàng từ lời giải các bài toán con; thường bài toán gốc chính là một trong số chúng (ở 6.1 là OPT(n)).

3. **Có thứ tự từ nhỏ đến lớn.** Có một thứ tự tự nhiên trên các bài toán con và một công thức truy hồi tính bài toán con từ các bài toán con "nhỏ hơn" (ở 6.1 là thứ tự j = 0, 1, …, n).

Con gà và quả trứng

Sách lưu ý một nghịch lý: ta không thể chắc một tập bài toán con là tốt cho đến khi tìm ra công thức truy hồi nối chúng lại, nhưng lại khó tìm công thức nếu chưa có tập bài toán con. Trên thực tế, hai thứ được tìm **cùng lúc**, qua thử và sửa. Một câu hỏi mồi hiệu quả là: **"Phần tử cuối cùng đóng vai trò gì trong lời giải tối ưu?"** Ở 6.1, câu trả lời là "có hoặc không có"; ở 6.3 sẽ là "thuộc đoạn bắt đầu ở đâu"; ở 6.4 sẽ là "có hoặc không, và còn lại bao nhiêu sức chứa".

6.3 Bình phương tối thiểu từng đoạn: lựa chọn nhiều nhánh

Ở 6.1 mỗi bước chỉ có hai lựa chọn (nhận hay bỏ). Mục này đưa ra bài toán mà bước cuối có tới **j lựa chọn**, nên công thức truy hồi lấy min trên j khả năng; thời gian tăng lên $O(n^2)$ nhưng ý tưởng giữ nguyên.

Nhắc lại: hồi quy tuyến tính một đường thẳng

Cho tập điểm $P = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ với $x_1 < x_2 < \ldots < x_n$. Với đường thẳng $L: y = ax + b$, sai số của L trên P là tổng bình phương khoảng cách theo phương đứng:

```
Error(L, P) = Σ (y_i − a·x_i − b)²        (tổng lấy theo i = 1, ..., n)
```

Đường thẳng có sai số nhỏ nhất có công thức đóng (lấy đạo hàm theo a, b rồi cho bằng 0):

```
a = [ n·Σ x_i·y_i − (Σ x_i)·(Σ y_i) ] / [ n·Σ x_i² − (Σ x_i)² ]

b = ( Σ y_i − a·Σ x_i ) / n
```

Khi một đường thẳng là không đủ

Nhiều chuỗi dữ liệu thay đổi xu hướng theo từng giai đoạn. Ví dụ một chuỗi mực nước hoặc lưu lượng có thể tăng đều, sau đó đi ngang khi hồ chứa đầy, rồi giảm khi xả. Ép một đường thẳng duy nhất sẽ cho sai số lớn; còn cho phép tuỳ ý nhiều đường thì mỗi cặp điểm liền kề một đoạn sẽ cho sai số 0 — vô nghĩa vì đó là quá khớp.

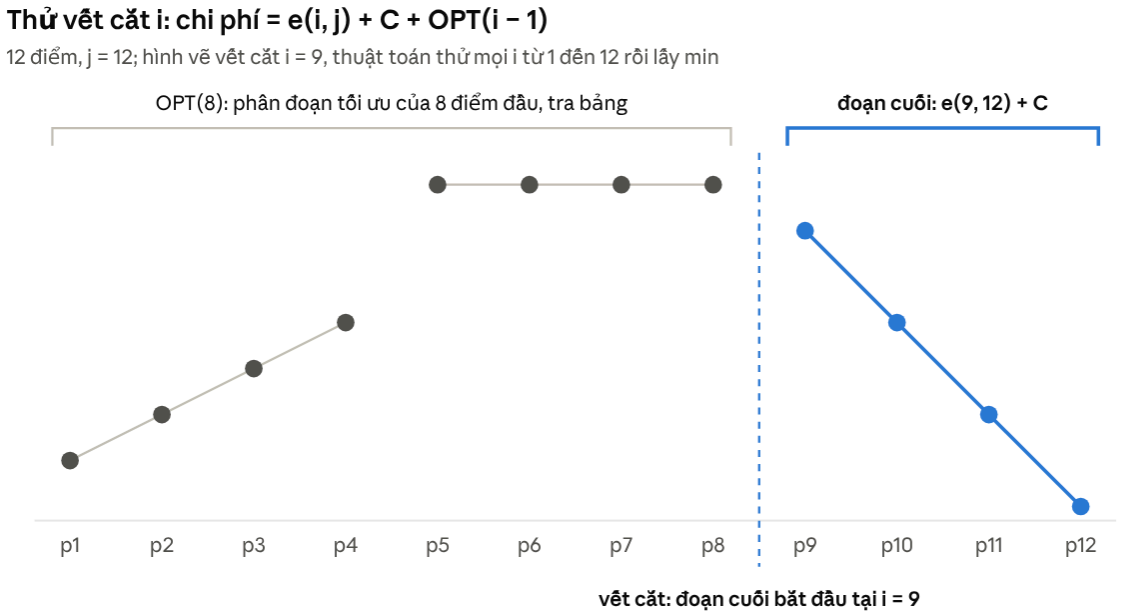
Sách giải quyết bằng **hàm phạt**. Ta chia dãy điểm thành các đoạn liên tiếp, mỗi đoạn khớp một đường thẳng tối ưu. Nếu có L đoạn thì:

```
Penalty = L·C + (tổng sai số của tất cả các đoạn)
```

Hăng số C > 0 là **"giá của một đoạn"**: C nhỏ thì thuật toán sẵn sàng dùng nhiều đoạn để giảm sai số; C lớn thì nó tiết kiệm đoạn và chấp nhận sai số. Mục tiêu là tìm phân đoạn có Penalty nhỏ nhất.

Thử nghiệm với 12 điểm x = 1, …, 12 và y = 1, 2, 3, 4, 7, 7, 7, 7, 6, 4, 2, 0 (tăng, đi ngang, giảm), kết quả chạy chương trình ở cuối mục cho thấy rõ vai trò của C:

C	Phân đoạn tối ưu	Penalty
1	{1–4}, {5–8}, {9–12}	3,00
10	{1–7}, {8–12}	23,54
200	{1–12}	273,60



Hình 6.3 · một vết cắt trong công thức truy hồi

Với mỗi vị trí cắt i, phần bên phải là một đường thẳng duy nhất (tính sẵn e(i, j)), phần bên trái đã có đáp án tối ưu trong bảng. Thuật toán chỉ việc cộng hai phần với C và so sánh j vết cắt.

Quan sát then chốt: điểm cuối thuộc đoạn nào

Điểm cuối p_n chắc chắn thuộc **đúng một đoạn**, và đoạn đó bắt đầu tại một điểm p_i nào đó ($1 \leqslant i \leqslant n$). Nếu biết i, bài toán tách làm hai phần độc lập:

- Đoạn cuối $\{p_i, \ldots, p_n\}$ góp chi phí $e_{i,n} + C$, trong đó $e_{i,n}$ là sai số nhỏ nhất của một đường thẳng khớp các điểm p_i đến p_n.
- Các điểm còn lại $\{p_1, \ldots, p_{i-1}\}$ phải được phân đoạn tối ưu, góp chi phí OPT(i−1).

Vì không biết i, ta **thử mọi i và lấy min**. Hình ảnh: bạn cầm một chiếc kéo, thử cắt đoạn cuối ở từng vị trí có thể; với mỗi vết cắt, phần bên trái đã có sẵn đáp án tối ưu trong bảng.

Công thức truy hồi

Gọi OPT(j) là chi phí tối ưu cho các điểm $p_1, \ldots, p_j$, với OPT(0) = 0:

```
OPT(j) = min   ( e(i, j) + C + OPT(i − 1) )        với OPT(0) = 0
        1≤i≤j
```

Đoạn $\{p_i, \ldots, p_j\}$ được dùng trong một lời giải tối ưu cho bài toán con j khi và chỉ khi min đạt tại chỉ số i đó. Đây là chìa khoá để truy vết.

Thuật toán

```
Segmented-Least-Squares(n)
  M[0] ← 0
  for mọi cặp i ≤ j: tính e[i, j]
  for j = 1 to n
    M[j] ← min over 1 ≤ i ≤ j of ( e[i, j] + C + M[i-1] )
  return M[n]

Find-Segments(j)
  if j = 0 then dừng
  else
    tìm i làm min của e[i, j] + C + M[i-1]
    xuất đoạn {p_i, ..., p_j};  Find-Segments(i-1)
```

Phân tích độ phức tạp

Công việc	Số lượng	Chi phí mỗi việc	Tổng
Tính mọi $e_{i,j}$ (cách trực tiếp)	$O(n^2)$ cặp	$O(n)$	$O(n^3)$
Tính mọi $e_{i,j}$ (dùng tổng tiền tố Σx, Σy, Σx^2, Σxy, Σy^2)	$O(n^2)$ cặp	$O(1)$	$O(n^2)$
Điền bảng M	n ô	O(n) phép so sánh	$O(n^2)$
Truy vết Find-Segments	tối đa n bước	$O(n)$	$O(n^2)$

Sách trình bày bước tính $e_{i,j}$ là $O(n^3)$ rồi lưu ý có thể giảm xuống $O(n^2)$. Bảng chỉ có n+1 ô, nhưng **mỗi ô cần xét tới j ô trước nó** — đó là cái giá của lựa chọn nhiều nhánh.

Cài đặt Python ($O(n^2)$ nhờ tổng tiền tố)

```
def segmented_least_squares(pts, C):
    """pts: danh sách (x, y). Trả về (Penalty tối ưu, danh sách đoạn (i, j),
đánh số từ 1)."""
    pts = sorted(pts); n = len(pts)
    Sx, Sy, Sxx, Sxy, Syy = ([0.0] * (n + 1) for _ in range(5))
    for k, (x, y) in enumerate(pts, 1):
        Sx[k] = Sx[k-1] + x;     Sy[k] = Sy[k-1] + y
        Sxx[k] = Sxx[k-1] + x*x; Sxy[k] = Sxy[k-1] + x*y; Syy[k] = Syy[k-1] +
y*y

    def err(i, j):                      # e(i, j) trong O(1)
        m = j - i + 1
        sx, sy = Sx[j] - Sx[i-1], Sy[j] - Sy[i-1]
        sxx, sxy, syy = Sxx[j] - Sxx[i-1], Sxy[j] - Sxy[i-1], Syy[j] - Syy[i-
1]
        den = m * sxx - sx * sx
        a = 0.0 if den == 0 else (m * sxy - sx * sy) / den
        b = (sy - a * sx) / m
        return max(0.0, syy + a*a*sxx + m*b*b - 2*a*sxy - 2*b*sy + 2*a*b*sx)

    M, arg = [0.0] * (n + 1), [0] * (n + 1)
    for j in range(1, n + 1):
        M[j], arg[j] = min((err(i, j) + C + M[i-1], i) for i in range(1, j +
1))

    segs, j = [], n
    while j > 0:
        i = arg[j]; segs.append((i, j)); j = i - 1
    return M[n], segs[::-1]

pts = list(zip(range(1, 13), [1, 2, 3, 4, 7, 7, 7, 7, 6, 4, 2, 0]))
print(segmented_least_squares(pts, C=1))
# (3.0, [(1, 4), (5, 8), (9, 12)])
```

6.4 Tổng tập con và cái ba lô: thêm một biến

Mục này cho thấy khi bài toán con một chiều OPT(i) không đủ thông tin, ta **thêm một biến** — sức chứa còn lại w — để có bảng hai chiều OPT(i, w). Kết quả là thuật toán O(nW), một thời gian chạy "giả đa thức".

Phát biểu hai bài toán

- **Tổng tập con (Subset Sum).** Có n vật, vật i có trọng lượng w_i (số nguyên không âm), và một giới hạn W. Chọn tập con S sao cho tổng trọng lượng ⩽ W và **lớn nhất có thể**. Sách mô tả bằng một máy chỉ chạy được W đơn vị thời gian, mỗi việc i cần w_i đơn vị; muốn máy bận càng nhiều càng tốt.
- **Cái ba lô (Knapsack).** Mỗi vật có thêm giá trị v_i. Chọn tập con có tổng trọng lượng ⩽ W và **tổng giá trị lớn nhất**. Subset Sum là trường hợp riêng khi $v_i = w_i$.

Hình ảnh: một xe chở vật tư cho công trình có tải trọng W tấn; mỗi kiện hàng nặng w_i tấn và có mức cần thiết v_i. Xếp kiện nào lên xe để tổng mức cần thiết lớn nhất mà xe không quá tải?

Vì sao tham lam thất bại

- Xếp vật **nặng trước**: với trọng lượng {W/2 + 1, W/2, W/2}, tham lam lấy W/2 + 1 rồi không lấy thêm được gì, trong khi hai vật W/2 lấp đầy đúng W.
- Xếp vật **nhẹ trước**: với {1, W/2, W/2}, tham lam lấy 1 và một vật W/2, được W/2 + 1, trong khi tối ưu là W.

Thử dùng bài toán con một chiều — và vì sao không đủ

Bắt chước 6.1, ta thử định nghĩa OPT(i) là giá trị tốt nhất khi dùng các vật 1, …, i. Nếu vật n không được chọn thì OPT(n) = OPT(n−1), ổn. Nhưng nếu **vật n được chọn**, ta không thể chỉ nói "phần còn lại là OPT(n−1)": sau khi đặt vật n vào, sức chứa chỉ còn $W - w_n$, và OPT(n−1) không biết điều đó. Ở 6.1, nhận yêu cầu n chỉ loại bỏ một **khối liên tiếp** yêu cầu; ở đây, nhận vật n làm thay đổi **ràng buộc** cho mọi vật còn lại. Bài toán con phải mang theo thông tin về sức chứa còn lại.

Thêm biến w: bài toán con hai chiều

Định nghĩa OPT(i, w) = giá trị tốt nhất khi chỉ dùng các vật 1, …, i và sức chứa w, với 0 ⩽ i ⩽ n, 0 ⩽ w ⩽ W. Đáp án là OPT(n, W). Xét vật i:

- Nếu $w_i > w$: vật i không vừa, buộc phải bỏ, nên OPT(i, w) = OPT(i−1, w).
- Ngược lại, chọn cái tốt hơn giữa **bỏ** vật i và **nhận** vật i (sức chứa giảm còn $w - w_i$):

```
OPT(i, w) = OPT(i − 1, w)                                  nếu w_i > w
OPT(i, w) = max( OPT(i − 1, w) ,  w_i + OPT(i − 1, w − w_i) )  ngược lại
```

với OPT(0, w) = 0 cho mọi w. Hình ảnh: mỗi ô của bảng là một câu hỏi "nếu chỉ có i vật đầu và chiếc ba lô w ki-lô-gam thì làm được tốt nhất bao nhiêu?". Để trả lời, ô (i, w) nhìn lên **ô ngay phía trên** (bỏ vật i) và **ô phía trên, lùi sang trái w_i cột** (nhận vật i).

Thuật toán Subset-Sum

```
Subset-Sum(n, W)
  mảng M[0..n, 0..W];  M[0, w] ← 0 với mọi w
  for i = 1 to n
    for w = 0 to W
      if w_i > w then M[i, w] ← M[i-1, w]
      else M[i, w] ← max( M[i-1, w],  w_i + M[i-1, w - w_i] )
  return M[n, W]
```

Bảng được điền **từng hàng, từ trên xuống**; mỗi ô chỉ dùng hai ô ở hàng ngay trên nên luôn đã có giá trị.

Chạy tay: W = 6, ba vật nặng 2, 2, 3

Hàng i (vật)	w = 0	1	2	3	4	5	6
i = 0 (chưa có vật)	0	0	0	0	0	0	0
i = 1 ($w_1 = 2$)	0	0	2	2	2	2	2
i = 2 ($w_2 = 2$)	0	0	2	2	4	4	4
i = 3 ($w_3 = 3$)	0	0	2	3	4	5	**5**

Đọc vài ô tiêu biểu:

- M[3, 5] = max(M[2, 5], 3 + M[2, 2]) = max(4, 3 + 2) = **5**: nhận vật 3 tốt hơn.
- M[3, 4] = max(M[2, 4], 3 + M[2, 1]) = max(4, 3 + 0) = **4**: bỏ vật 3 tốt hơn.
- M[3, 6] = max(M[2, 6], 3 + M[2, 3]) = max(4, 3 + 2) = **5**.

Đáp án OPT(3, 6) = 5. **Truy vết** từ ô (3, 6): M[3, 6] = 5 ≠ M[2, 6] = 4 nên vật 3 được nhận, chuyển sang ô (2, 6 − 3) = (2, 3). M[2, 3] = 2 = M[1, 3] nên bỏ vật 2, chuyển lên (1, 3). M[1, 3] = 2 ≠ M[0, 3] = 0 nên nhận vật 1. Tập tối ưu là {1, 3}, tổng 2 + 3 = 5. Truy vết chỉ đi qua n hàng nên tốn O(n).

Độ phức tạp giả đa thức

Bảng có (n+1)(W+1) ô, mỗi ô tính trong O(1), nên Subset-Sum chạy trong **O(nW)**. Nghe như đa thức, nhưng có một cái bẫy:

- Kích thước đầu vào tính bằng **số bit**, và W chỉ cần khoảng $\log_2 W$ bit để viết ra.
- Do đó O(nW) là **hàm mũ theo số bit của W**. Với $W = 10^9$, bảng có tới một tỷ cột dù W chỉ là một con số 10 chữ số.
- Thuật toán như vậy gọi là **giả đa thức** (pseudo-polynomial): nhanh khi W nhỏ, chậm khi W lớn.

Điều này phù hợp với thực tế rằng Subset Sum và Knapsack là các bài toán NP-đầy đủ (Chương 8). Sách cũng quay lại Knapsack ở Chương 11, dùng chính ý tưởng quy hoạch động để xây dựng thuật toán xấp xỉ.

Mở rộng sang Knapsack

Chỉ cần thay "w_i +" bằng "v_i +" trong vế nhận:

```
OPT(i, w) = OPT(i − 1, w)                                  nếu w_i > w
OPT(i, w) = max( OPT(i − 1, w) ,  v_i + OPT(i − 1, w − w_i) )  ngược lại
```

Thuật toán, cách truy vết và thời gian O(nW) giữ nguyên.

```
def knapsack(items, W):
    """items: danh sách (w_i, v_i), w_i nguyên. Trả về (giá trị tối ưu, các
vật được chọn, đánh số từ 1)."""
    n = len(items)
    M = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w_i, v_i = items[i - 1]
        for w in range(W + 1):
            M[i][w] = M[i - 1][w]                                  # bỏ vật
i
            if w_i <= w and v_i + M[i - 1][w - w_i] > M[i][w]:
                M[i][w] = v_i + M[i - 1][w - w_i]                  # nhận
vật i
    chosen, w = [], W
    for i in range(n, 0, -1):                                      # truy
vết
        if M[i][w] != M[i - 1][w]:
            chosen.append(i); w -= items[i - 1][0]
    return M[n][W], chosen[::-1]

# 5 vật (trọng lượng, giá trị), sức chứa 11
print(knapsack([(1, 1), (2, 6), (5, 18), (6, 22), (7, 28)], 11))
# (40, [3, 4])  -> vật 3 và 4: nặng 5 + 6 = 11, giá trị 18 + 22 = 40
```

Ví dụ trên cũng là một phản ví dụ cho tham lam theo tỉ lệ giá trị/trọng lượng: vật 5 có tỉ lệ cao nhất (4,0), nhưng lời giải tối ưu lại không dùng nó.

Tổng kết: bốn mục, một công thức tư duy

Cả bốn mục đều trả lời cùng một câu hỏi — "phần tử cuối cùng đóng vai trò gì trong lời giải tối ưu?" — và khác nhau ở số nhánh lựa chọn và số chiều của bài toán con.

Mục	Bài toán con	Số lựa chọn mỗi bước	Kích thước bảng	Thời gian	Bài học
6.1 Lập lịch có trọng số	OPT(j): dùng yêu cầu 1..j	2 (nhận / bỏ)	n+1	O(n) sau khi sắp xếp O(n log n)	Ghi nhớ biến hàm mũ thành tuyến tính
6.2 Nguyên lý	—	—	—	—	Lặp dưới lên tương đương ghi nhớ; ba điều kiện của bài toán con
6.3 Bình phương tối thiểu từng đoạn	OPT(j): điểm 1..j	j (đoạn cuối bắt đầu ở đâu)	n+1	$O(n^2)$, hoặc $O(n^3)$ nếu tính $e_{i,j}$ trực tiếp	Truy hồi lấy min trên nhiều nhánh
6.4 Subset Sum / Knapsack	OPT(i, w): vật 1..i, sức chứa w	2 (nhận / bỏ)	(n+1) (W+1)	O(nW), giả đa thức	Thêm biến khi bài toán con thiếu thông tin

Bốn bước thiết kế một thuật toán quy hoạch động

1. **Định nghĩa bài toán con** bằng lời, thật chính xác: OPT(·) nghĩa là gì, tham số là gì.
2. **Viết công thức truy hồi** bằng cách hỏi về phần tử cuối: có những khả năng nào, mỗi khả năng quy về bài toán con nào.
3. **Chọn thứ tự điền bảng** sao cho mỗi ô chỉ dùng các ô đã điền; xác định các ô cơ sở.
4. **Truy vết** từ ô đáp án ngược về các ô cơ sở để dựng lại lời giải, rồi phân tích thời gian = số ô × chi phí mỗi ô.

Câu hỏi tự kiểm tra

1. Vì sao Compute-Opt chạy trong thời gian hàm mũ, trong khi chỉ có n+1 bài toán con khác nhau?
   a. *Gợi ý:* cây gọi lặp lại cùng một OPT(k) nhiều lần; với p(j) = j−2 số lời gọi tăng như dãy Fibonacci.
2. Trong ví dụ 6 yêu cầu, nếu v_6 đổi từ 1 thành 3 thì lời giải tối ưu thay đổi thế nào?
   a. *Gợi ý:* v_6 + M[3] = 3 + 6 = 9 > M[5] = 8, nên nhận 6; lời giải thành {1, 3, 6} với giá trị 9.
3. Ở 6.3, điều gì xảy ra với số đoạn khi C → 0 và khi C → ∞?
   a. *Gợi ý:* C → 0 cho nhiều đoạn đến mức sai số gần 0; C → ∞ ép về một đường thẳng duy nhất.
4. Vì sao OPT(i) một chiều không đủ cho Subset Sum, trong khi lại đủ cho 6.1?
   a. *Gợi ý:* nhận vật n làm giảm sức chứa cho mọi vật còn lại, còn nhận yêu cầu n chỉ loại bỏ một khối liên tiếp.
5. Thuật toán O(nW) có phải là thuật toán đa thức không? Vì sao?

a. *Gợi ý:* không; W chỉ chiếm khoảng $\log_2 W$ bit, nên O(nW) là hàm mũ theo kích thước đầu vào — gọi là giả đa thức.